When personalization deters delegation to algorithms — Replication analysis

Lara Suraci & Irenaeus Wolff

09 Oktober 2026

Contents

1 Data import and preparation **3**
1.1 Shared plotting constants . . . 3
1.2 Load raw data . . . 3
1.3 Preregistered exclusions . . . 3
1.4 Build participant-level Part-1 preference measures . . . 4

2 Descriptive overview **6**
2.1 Choice quality and payoffs . . . 6
2.2 Round-level delegation profile . . . 6
2.3 Figure 1 — Average delegation over decisions . . . 7

3 Primary test 1 — First-contact delegation (round 1) **9**
3.1 Effect size and CI for the round-1 contrast . . . 9

4 Primary test 2 — Delegation across repeated decisions **11**
4.1 Participant-level Wilcoxon–Mann–Whitney test . . . 11
4.2 Effect size and CI for the WMW test . . . 11
4.3 Risk aversion . . . 11
4.4 Figure 3 — ECDF of participant-level average delegation . . . 12
4.5 Preregistered regression specification . . . 13
4.6 Robustness check: logistic regression . . . 14
4.7 Supporting regression . . . 15

5 Categorical non-delegation **16**
5.1 Significance test and effect size for categorical non-delegation . . . 17
5.2 Comparisons of never-delegators vs. delegators . . . 18
5.3 Choice quality among never-delegators (new Limitations check) . . . 21

6 Responses to feedback (win-stay / lose-shift) **23**

7 Exploratory mechanisms **25**
7.1 Coefficient model (interaction with performance expectations) . . . 25
7.2 Treatment comparison of expected algorithmic errors . . . 26

8 Post-task reasons **28**
8.1 Figure 4 — Reasons by treatment and delegation type . . . 28
8.2 Aggregate reason patterns . . . 29
8.3 LPM: never-delegation on reasons . . . 30

9 Exclusion-criterion breakdown (Method section) 32

10 Power analysis (pilot-based, new Limitations paragraph) 34
10.1 Minimum detectable effect (MDE) at 80% power 34
10.2 MDE and power for the average-delegation comparison 35
10.3 Power to re-detect the pilot's observed effect 35

```
library(dplyr)
library(tidyr)
library(ggplot2)
library(scales)
library(Exact)      # Boschloo exact test
library(fixest)     # LPM with fixed effects + clustered SE
library(lmtest)     # robust coefficient tests
library(sandwich)   # HC2 standard errors
library(texreg)     # regression tables
# library(readODS) # only needed for the Appendix task-matrix chunk near the end
```

1 Data import and preparation

1.1 Shared plotting constants

To keep figures visually consistent we fix the treatment colour palette once.

```
# Treatment colours used throughout (Pref = preference-based, EV = optimisation)
col_pref <- "#4E79A7"
col_ev   <- "#D55E00"
algo_cols <- c("Pref" = col_pref, "EV" = col_ev)
```

1.2 Load raw data

Paths are relative to this script's location (replication-package layout).

```
choices <- read.delim("260206_1528_537_lotteryChoices.xls")
subjects <- read.csv("allSubjects.csv", header = TRUE)
prolificData <- read.csv("prolific_demographic_export_approved.csv")

choicesP1 <- subset(choices, Part == 1)
```

1.3 Preregistered exclusions

We apply the preregistered exclusion rules: failed bot/slider check at first attempt, failed comprehension checks after the second attempt, failure to confirm careful participation (`Reliable < 6`), disclosed AI usage, incomplete data (`Delegated < 0`), and the log-time "too fast" rule.

```
# "Too fast" flag based on the preregistered log-completion-time rule
prolificData$logCompleTime <- log(prolificData$Time.taken)
prolificData$tooFast <- 1 * (
  prolificData$logCompleTime <
    mean(log(prolificData$Time.taken)) - 2 * sd(log(prolificData$Time.taken))
)

keep_ids <- prolificData$Participant.id[prolificData$tooFast == 0]

choices <- subset(
  choices,
  BotCheck1attempt == 1 &
    Part1CQsPassedOnAttempt <= 2 &
    Part2CQsPassedOnAttempt <= 2 &
    Reliable >= 6 &
    AIUsageInStudy == 0 &
    Delegated >= 0 &
    ProlificID %in% keep_ids
)

subjects  <- subset(subjects,  ProlificID %in% choices$ProlificID)
choicesP1 <- subset(choicesP1, ProlificID %in% choices$ProlificID)

# Final analysis sample (paper: N = 233; EV = 118, Pref = 115)
cat("Final sample size per condition (1 = EV, 0 = Preference-based):\n")
```

```
## Final sample size per condition (1 = EV, 0 = Preference-based):
```

```r
table(subjects$EVAlgorithm)
```

```
##
##   0   1
## 115 118
```

```r
# Convenience label for the algorithm condition
choices <- choices %>%
  mutate(Algorithm = ifelse(EVAlgorithm == 1, "EV", "Pref"))
```

1.4 Build participant-level Part-1 preference measures

These variables (revealed risk preferences from Part 1, per-task confidence, and post-task expectations) are merged onto the decision-level `choices` data for use in the regressions and the appendix coefficient plot.

```r
# EV-maximising choice in each Part-1 task
choicesP1 <- choicesP1 %>%
  mutate(EVChoice = case_when(
    grepl("^P", ChoiceType) & PBetChosen == 1 ~ 1,
    grepl("^D", ChoiceType) & PBetChosen == 0 ~ 1,
    TRUE ~ 0
  ))

# "True" P-bet choice (less-risky choice when it was NOT EV-maximising)
choicesP1 <- choicesP1 %>%
  mutate(truePChoice = case_when(
    grepl("^D", ChoiceType) & PBetChosen == 1 ~ 1,
    TRUE ~ 0
  ))

# Participant-level Part-1 summaries
p1_summaries <- choicesP1 %>%
  group_by(ProlificID) %>%
  summarise(
    EVChoices    = sum(EVChoice),
    truePChoices = sum(truePChoice),
    PBet_count   = sum(PBetChosen),
    .groups = "drop"
  )

choices <- choices %>%
  left_join(p1_summaries, by = "ProlificID") %>%
  # per-task Part-1 confidence (matched on participant x option set)
  left_join(
    choicesP1 %>% select(ProlificID, OptionsetID, ConfidenceP1 = Confidence),
    by = c("ProlificID", "OptionsetID")
  ) %>%
  # participant-level demographics and post-task expectations
  left_join(
    subjects %>% select(
      ProlificID, Female, Age, Education, HHIncome,
      ProbOfBadComputerChoices, ProbOfAbsurdComputerChoices, ComputerRiskTaking
    ),
    by = "ProlificID"
```

```
)
```

2 Descriptive overview

2.1 Choice quality and payoffs

Choice quality among non-delegated decisions and the payoff advantage of delegation (paper, Section "Primary test: first-contact delegation").

```
# Share of non-dominated self-chosen options, by condition
choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm) %>%
  summarise(
    avNonDom = mean(Nondominated[Delegated == 0 & Option > 0], na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 2
##   Algorithm avNonDom
##   <chr>        <dbl>
## 1 EV           0.341
## 2 Pref         0.351
```

```
# Average payoff: delegated vs. non-delegated decisions
# (paper: 243.49 points delegated vs. 195.18 points non-delegated, ~25% higher)
choices %>%
  filter(Part == 2) %>%
  group_by(Delegated) %>%
  summarise(meanPayoff = mean(RelevantPayoff, na.rm = TRUE), .groups = "drop")
```

```
## # A tibble: 2 x 2
##   Delegated meanPayoff
##       <int>      <dbl>
## 1         0       195.
## 2         1       243.
```

2.2 Round-level delegation profile

```
choicesAv <- choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm, WithinPartDecisionNumber) %>%
  summarise(
    avDeleg  = mean(Delegated, na.rm = TRUE),
    avNonDom = mean(Nondominated[Delegated == 0 & Option > 0], na.rm = TRUE),
    delegSE = sd(Delegated, na.rm = TRUE) / sqrt(n()),
    .groups = "drop"
  )
choicesAv
```

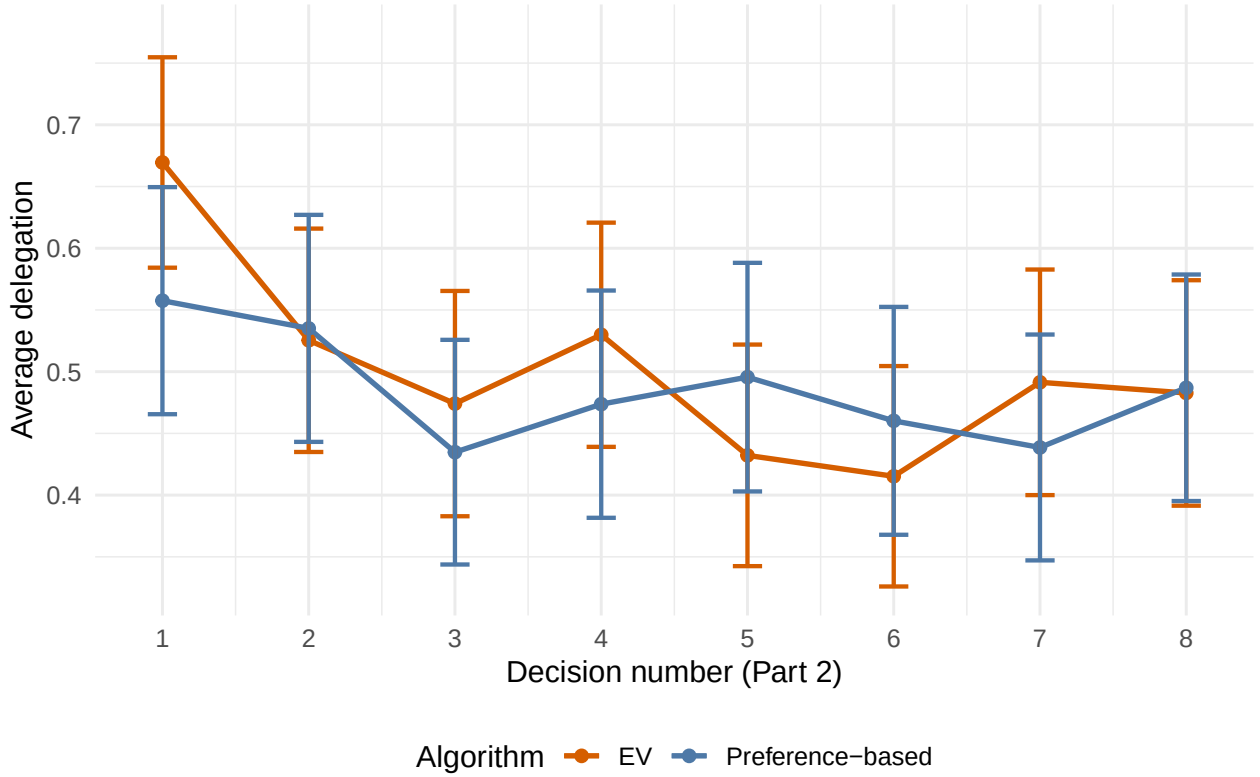
```
## # A tibble: 16 x 5
##    Algorithm WithinPartDecisionNumber avDeleg avNonDom delegSE
##    <chr>                        <int>   <dbl>    <dbl>   <dbl>
##  1 EV                               1   0.669    0.342  0.0435
##  2 EV                               2   0.525    0.214  0.0462
##  3 EV                               3   0.474    0.3    0.0466
##  4 EV                               4   0.530    0.333  0.0463
```

```
##  5 EV                              5   0.432    0.269  0.0458
##  6 EV                              6   0.415    0.464  0.0456
##  7 EV                              7   0.491    0.356  0.0466
##  8 EV                              8   0.483    0.433  0.0466
##  9 Pref                            1   0.558    0.4    0.0469
## 10 Pref                            2   0.535    0.327  0.0469
## 11 Pref                            3   0.435    0.344  0.0464
## 12 Pref                            4   0.474    0.267  0.0470
## 13 Pref                            5   0.496    0.429  0.0472
## 14 Pref                            6   0.460    0.333  0.0471
## 15 Pref                            7   0.439    0.391  0.0467
## 16 Pref                            8   0.487    0.322  0.0468
```

2.3 Figure 1 — Average delegation over decisions

```
fig1 <- choicesAv %>%
  mutate(Algorithm = factor(Algorithm, levels = c("EV", "Pref"),
                            labels = c("EV", "Preference-based"))) %>%
  ggplot(aes(x = WithinPartDecisionNumber, y = avDeleg,
             color = Algorithm, group = Algorithm)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  geom_errorbar(aes(ymin = avDeleg - 1.96 * delegSE, ymax = avDeleg + 1.96 * delegSE),
                width = 0.2, linewidth = 0.8) +
  scale_x_continuous(breaks = 1:8) +
  scale_y_continuous(breaks = (4:7)/10, limits = c(0.325, 0.775)) +
  scale_color_manual(values = c("EV" = col_ev, "Preference-based" = col_pref)) +
  labs(x = "Decision number (Part 2)",
       y = "Average delegation",
       color = "Algorithm") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

fig1
```




```
ggsave("Figure1_AverageDelegation.pdf", fig1, width = 7, height = 4.5)
```

3 Primary test 1 — First-contact delegation (round 1)

Preregistered one-sided Boschloo exact test on round-1 delegation (paper: 66.9% vs. 55.8%, p = 0.0413).

```
tab <- table(
  Delegated   = choices$Delegated[choices$Part == 2 & choices$WithinPartDecisionNumber ==
  ↪  1],
  EVAlgorithm = choices$EVAlgorithm[choices$Part == 2 & choices$WithinPartDecisionNumber
  ↪  == 1]
)

# Delegation share by condition (column 2 = EV)
prop.table(tab, margin = 2)
```

```
##          EVAlgorithm
## Delegated         0         1
##         0 0.4424779 0.3305085
##         1 0.5575221 0.6694915
```

```
# One-sided Boschloo exact test (H1: EV > Pref)
exact.test(tab, method = "Boschloo", alternative = "greater", to.plot = FALSE)$p.value
```

```
## [1] 0.0412565
```

3.1 Effect size and CI for the round-1 contrast

Risk difference (Wald 95% CI) and odds ratio (Fisher's exact two-sided 95% CI), as reported in the manuscript's new Table 1 (RD = 11.1pp [-1.3, 23.7]; OR = 1.60 [0.91, 2.84]).

```
p_ev   <- prop.table(tab, margin = 2)["1", "1"]
p_pref <- prop.table(tab, margin = 2)["1", "0"]
n_ev   <- sum(tab[, "1"])
n_pref <- sum(tab[, "0"])

# Risk difference + Wald 95% CI
rd <- p_ev - p_pref
se_rd <- sqrt(p_ev * (1 - p_ev) / n_ev + p_pref * (1 - p_pref) / n_pref)
cat("Risk difference:", round(rd, 4), "\n")
```

```
## Risk difference: 0.112
```

```
cat("Wald 95% CI:", round(rd - 1.96 * se_rd, 4), "to", round(rd + 1.96 * se_rd, 4), "\n")
```

```
## Wald 95% CI: -0.0129 to 0.2368
```

```
# Odds ratio + Fisher's exact two-sided 95% CI
fisher.test(tab)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  tab
## p-value = 0.1044
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.9109897 2.8414026
## sample estimates:
```

```
## odds ratio
##   1.604278
```

4 Primary test 2 — Delegation across repeated decisions

4.1 Participant-level Wilcoxon–Mann–Whitney test

Preregistered one-sided WMW test on participant-average delegation (paper: W = 6428, p = 0.2425).

```
delegAv <- choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm, ProlificID) %>%
  summarise(avDeleg = mean(Delegated, na.rm = TRUE), .groups = "drop") %>%
  mutate(Algorithm = factor(Algorithm, levels = c("Pref", "EV")))

wilcox.test(avDeleg ~ Algorithm, data = delegAv, alternative = "less")
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  avDeleg by Algorithm
## W = 6428, p-value = 0.2425
## alternative hypothesis: true location shift is less than 0
```

4.2 Effect size and CI for the WMW test

Rank-biserial correlation with a bootstrap 95% CI, as reported in the manuscript's new Table 1 ($r_{rb} \approx 0.05$, bootstrap CI [-0.09, 0.21]).

```
rank_biserial <- function(x, y) {
  n1 <- length(x); n2 <- length(y)
  U <- wilcox.test(x, y)$statistic
  1 - (2 * U) / (n1 * n2)
}

x_pref <- delegAv$avDeleg[delegAv$Algorithm == "Pref"]
x_ev   <- delegAv$avDeleg[delegAv$Algorithm == "EV"]

r_rb <- rank_biserial(x_pref, x_ev)
cat("Rank-biserial r:", round(r_rb, 4), "\n")
```

```
## Rank-biserial r: 0.0526
```

```
set.seed(1)
B <- 2000
boot_r <- replicate(B, {
  xb <- sample(x_pref, replace = TRUE)
  yb <- sample(x_ev,   replace = TRUE)
  rank_biserial(xb, yb)
})
cat("Bootstrap 95% CI:", round(quantile(boot_r, 0.025), 4), "to",
    round(quantile(boot_r, 0.975), 4), "\n")
```

```
## Bootstrap 95% CI: -0.1012 to 0.1981
```

4.3 Risk aversion

```
# calculate how often the EV-algorithm chooses p-bets
table(choicesP1$ChoiceType[choicesP1$OptionsetID <= 8] == "P105")
```

```
##
## FALSE  TRUE
##   932   932
```

```
# 50-50; contrast that to...
pBetChoices <- choicesP1 %>%
  filter(OptionsetID <= 8) %>% # among the relevant Option sets
  group_by(ProlificID) %>%
  summarise(mean(PBetChosen)) # how often did the participant choose a p-bet
prop.table(table(pBetChoices > 0.5))
```

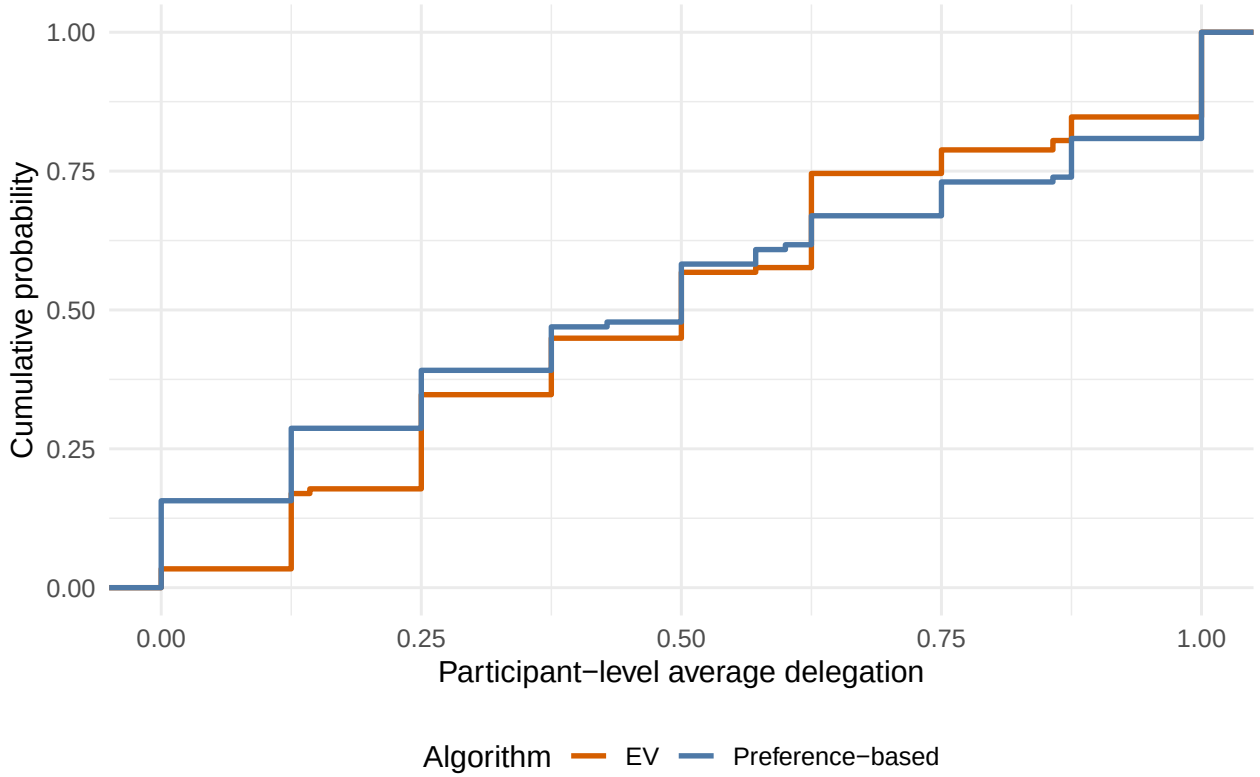
```
##
##     FALSE      TRUE
## 0.1030043 0.8969957
```

for 89.7% of the participants, the prefs-algorithm in expectation will produce fewer zero-outcomes than the EV-algorithm

4.4 Figure 3 — ECDF of participant-level average delegation

```
fig3 <- delegAv %>%
  mutate(Algorithm = factor(Algorithm, levels = c("EV", "Pref"),
                            labels = c("EV", "Preference-based"))) %>%
  ggplot(aes(x = avDeleg, color = Algorithm)) +
  stat_ecdf(linewidth = 1) +
  scale_color_manual(values = c("EV" = col_ev, "Preference-based" = col_pref)) +
  labs(x = "Participant-level average delegation",
       y = "Cumulative probability",
       color = "Algorithm") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

fig3
```




```
ggsave("Figure3_ECDF.pdf", fig3, width = 7, height = 4.5)
```

4.5 Preregistered regression specification

LPM with decision-set fixed effects and participant-clustered SEs. We first construct the feedback-history regressor (running share of zero outcomes among prior delegated choices).

```
choices <- choices %>%
  arrange(ProlificID, WithinPartDecisionNumber) %>%
  group_by(ProlificID) %>%
  mutate(
    delegated_round = (Delegated == 1),
    zero_payoff     = (RelevantPayoff == 0 & Delegated == 1),
    cum_delegated   = lag(cumsum(delegated_round), default = 0),
    cum_zero        = lag(cumsum(zero_payoff), default = 0),
    zero_rate       = ifelse(cum_delegated > 0, cum_zero / cum_delegated, NA)
  ) %>%
  ungroup()
```

```
# Preregistered model: treatment x (period + feedback history)
delRegPreReg <- feols(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + zero_rate) | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegPreReg)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,324
```

```
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                    Estimate Std. Error   t value   Pr(>|t|)
## EVAlgorithm                       -0.048717   0.083122 -0.586090 5.5845e-01
## WithinPartDecisionNumber          -0.001795   0.008257 -0.217380 8.2813e-01
## zero_rate                         -0.385350   0.078139 -4.931567 1.6664e-06
## EVAlgorithm:WithinPartDecisionNumber  0.001877   0.011504  0.163119 8.7058e-01
## EVAlgorithm:zero_rate             -0.049677   0.097673 -0.508609 6.1157e-01
##
## EVAlgorithm
## WithinPartDecisionNumber
## zero_rate                         ***
## EVAlgorithm:WithinPartDecisionNumber
## EVAlgorithm:zero_rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.467127     Adj. R2: 0.115951
##                  Within R2: 0.107265
```

4.6 Robustness check: logistic regression

Re-estimating the preregistered LPM as a logistic regression with the same fixed effects and cluster-robust SEs (paper: EVAlgorithm OR = 0.79, $p = 0.467$; zero-outcome-history OR = 0.19, $p < 0.001$).

```
delRegLogit <- feglm(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + zero_rate) | OptionsetID,
  family = "logit",
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegLogit)
```

```
## GLM estimation, family = binomial(link = "logit"), Dep. Var.: Delegated
## Observations: 1,324
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                    Estimate Std. Error   z value   Pr(>|z|)
## EVAlgorithm                       -0.240874   0.402319 -0.598714 5.4936e-01
## WithinPartDecisionNumber          -0.010738   0.037790 -0.284144 7.7630e-01
## zero_rate                         -1.661216   0.371278 -4.474320 7.6655e-06
## EVAlgorithm:WithinPartDecisionNumber  0.013504   0.053109  0.254272 7.9929e-01
## EVAlgorithm:zero_rate             -0.241165   0.474017 -0.508770 6.1091e-01
##
## EVAlgorithm
## WithinPartDecisionNumber
## zero_rate                         ***
## EVAlgorithm:WithinPartDecisionNumber
## EVAlgorithm:zero_rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood:  -829.7   Adj. Pseudo R2: 0.080391
##            BIC: 1,752.9     Squared Cor.: 0.125443
```

```
# Odds ratios
exp(coef(delRegLogit))
```

```
##                           EVAlgorithm             WithinPartDecisionNumber
##                             0.7859404                            0.9893196
##                             zero_rate EVAlgorithm:WithinPartDecisionNumber
##                             0.1899080                            1.0135957
##                EVAlgorithm:zero_rate
##                             0.7857117
```

4.7 Supporting regression

Plain treatment effect (all rounds; excluding round 1; round 1 only). The round-1-only LPM reproduces the Boschloo result ($\beta = 0.113$, one-sided $p = 0.041$).

```
# 1st-round regression in the spirit of the preregistered model: treatment (fixing period
↪  to 1, and taking out the [non-existing] feedback history)
delRegPreReg <- feols(
  Delegated ~ EVAlgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber == 1)
)
summary(delRegPreReg)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 231
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##             Estimate Std. Error t value Pr(>|t|)
## EVAlgorithm 0.112868     0.0648 1.74178 0.082884 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.474644     Adj. R2: 0.0145
##                  Within R2: 0.013716
```

5 Categorical non-delegation

We classify participants as never-, sometimes-, or always-delegators and compare never-delegation across conditions (paper: 15.7% Pref vs. 3.4% EV).

```
choices <- choices %>%
  group_by(ProlificID) %>%
  mutate(sumDelegations = sum(Delegated)) %>%
  ungroup() %>%
  left_join(
    prolificData %>% select(Participant.id, TimeInSeconds = Time.taken),
    by = c("ProlificID" = "Participant.id")
  )

analysis_data <- choices %>%
  group_by(ProlificID) %>%
  summarise(
    EVAlgorithm                = first(EVAlgorithm),
    sumDelegations             = first(sumDelegations),
    Female                     = first(Female),
    Age                        = first(Age),
    HHIncome                   = first(HHIncome),
    Education                  = first(Education),
    ProbOfBadComputerChoices   = first(ProbOfBadComputerChoices),
    ProbOfAbsurdComputerChoices = first(ProbOfAbsurdComputerChoices),
    ComputerRiskTaking         = first(ComputerRiskTaking),
    PBetChoices                = first(PBet_count),
    TimeInSeconds              = first(TimeInSeconds),
    .groups = "drop"
  ) %>%
  mutate(
    DelegationGroup = ifelse(sumDelegations == 0, "Never", "AtLeastOnce"),
    NonDelegator    = 1 * (sumDelegations == 0),
    DelegatorType   = factor(
      if_else(sumDelegations == 0, "NeverDelegator",
              if_else(sumDelegations == 8, "AlwaysDelegator", "SometimesDelegator")),
      levels = c("NeverDelegator", "SometimesDelegator", "AlwaysDelegator")
    )
  ) %>%
  left_join(
    subjects %>% select(ProlificID, RiskTaking, FinMarketExperience,
                        FinanciallyResponsible, AIXperience),
    by = "ProlificID"
  )

# Distribution of delegator types by condition (column 2 = EV)
table(analysis_data$DelegatorType, analysis_data$EVAlgorithm)
```

```
##
##                       0  1
##   NeverDelegator     18  4
##   SometimesDelegator 75 98
##   AlwaysDelegator    22 16
```

```r
prop.table(table(analysis_data$DelegatorType, analysis_data$EVAlgorithm), margin = 2)
```

```
##
##                              0          1
##   NeverDelegator     0.15652174 0.03389831
##   SometimesDelegator 0.65217391 0.83050847
##   AlwaysDelegator    0.19130435 0.13559322
```

5.1 Significance test and effect size for categorical non-delegation

Fisher's exact test, risk difference (Wald 95% CI), and odds ratio (Fisher's exact 95% CI), as newly added to the manuscript (paper: $p = 0.0014$; RD = 12.3pp [4.9, 19.7]; OR = 5.25 [1.65, 22.07]).

```r
nondeleg_tab <- table(
  NonDelegator = factor(analysis_data$NonDelegator, levels = c(1, 0)),
  EVAlgorithm  = factor(analysis_data$EVAlgorithm,  levels = c(0, 1))
)
nondeleg_tab   # rows: never-delegator yes/no; columns: 0 = Pref, 1 = EV
```

```
##             EVAlgorithm
## NonDelegator   0   1
##            1  18   4
##            0  97 114
```

```r
# Odds ratio of never-delegating, Pref relative to EV (expect OR ~ 5.25)
fisher.test(nondeleg_tab)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  nondeleg_tab
## p-value = 0.001436
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##   1.654958 22.069431
## sample estimates:
## odds ratio
##   5.254962
```

```r
p_pref_nd <- prop.table(nondeleg_tab, margin = 2)["1", "0"]
p_ev_nd   <- prop.table(nondeleg_tab, margin = 2)["1", "1"]
n_pref_nd <- sum(nondeleg_tab[, "0"])
n_ev_nd   <- sum(nondeleg_tab[, "1"])
# (row "1" = never-delegator; column "0" = Pref, "1" = EV)

rd_nd <- p_pref_nd - p_ev_nd
se_rd_nd <- sqrt(p_pref_nd * (1 - p_pref_nd) / n_pref_nd + p_ev_nd * (1 - p_ev_nd) /
↪  n_ev_nd)
cat("Risk difference:", round(rd_nd, 4), "\n")
```

```
## Risk difference: 0.1226
```

```r
cat("Wald 95% CI:", round(rd_nd - 1.96 * se_rd_nd, 4), "to",
    round(rd_nd + 1.96 * se_rd_nd, 4), "\n")
```

```
## Wald 95% CI: 0.0486 to 0.1966
```

5.2 Comparisons of never-delegators vs. delegators

The following tests reproduce the in-text comparisons (age, income, education, time, gender, risk-taking, reliability, AI experience, Part-1 satisfaction).

```
# Age
t.test(Age ~ DelegationGroup, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  Age by DelegationGroup
## t = 0.32876, df = 24.878, p-value = 0.7451
## alternative hypothesis: true difference in means between group AtLeastOnce and group Never is not eq
## 95 percent confidence interval:
##  -5.689245  7.849952
## sample estimates:
## mean in group AtLeastOnce       mean in group Never
##                  42.85308                  41.77273
```

```
# Time in experiment
t.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  TimeInSeconds by NonDelegator
## t = -0.9713, df = 25.374, p-value = 0.3406
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -385.8898  138.4297
## sample estimates:
## mean in group 0 mean in group 1
##        1113.043        1236.773
```

```
wilcox.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  TimeInSeconds by NonDelegator
## W = 1833, p-value = 0.1052
## alternative hypothesis: true location shift is not equal to 0
```

```
ks.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
##  Exact two-sample Kolmogorov-Smirnov test
##
## data:  TimeInSeconds by NonDelegator
## D = 0.28264, p-value = 0.06603
## alternative hypothesis: two-sided
```

```
# Household income (categorical)
chisq.test(table(analysis_data$DelegationGroup, analysis_data$HHIncome))
```

```
##
```

```
##  Pearson's Chi-squared test
##
## data:  table(analysis_data$DelegationGroup, analysis_data$HHIncome)
## X-squared = 13.171, df = 11, p-value = 0.2823
```

```
# Education (categorical)
chisq.test(table(analysis_data$DelegationGroup, analysis_data$Education))
```

```
##
##  Pearson's Chi-squared test
##
## data:  table(analysis_data$DelegationGroup, analysis_data$Education)
## X-squared = 1.859, df = 5, p-value = 0.8683
```

```
# Gender: never-delegation rate among males vs. females (Boschloo)
female_table <- table(
  DelegationGroup = analysis_data$DelegationGroup,
  Female          = analysis_data$Female
)
exact.test(female_table[, c(2, 4)], method = "Boschloo", to.plot = FALSE)$p.value
```

```
## [1] 0.1561955
```

```
# Self-reported financial risk-taking, reliability, AI experience
t.test(RiskTaking  ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  RiskTaking by NonDelegator
## t = -0.22379, df = 24.99, p-value = 0.8247
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -1.540832  1.238807
## sample estimates:
## mean in group 0 mean in group 1
##        4.530806        4.681818
```

```
t.test(FinanciallyResponsible ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  FinanciallyResponsible by NonDelegator
## t = 1.229, df = 24.512, p-value = 0.2307
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.6803629  2.6889799
## sample estimates:
## mean in group 0 mean in group 1
##        6.549763        5.545455
```

```
t.test(AIXperience ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
```

```
##
## data:  AIXperience by NonDelegator
## t = 0.86582, df = 24.274, p-value = 0.3951
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.9475606  2.3185214
## sample estimates:
## mean in group 0 mean in group 1
##        5.867299        5.181818
```

5.2.1 Non-parametric robustness check (Wilcoxon) for the comparisons above

As newly added to the manuscript footnote (paper: age $p = 0.649$; time $p = 0.105$; risk-taking $p = 0.854$; reliability $p = 0.956$; AI experience $p = 0.445$). Note: the manuscript's "reliability" item corresponds to `Reliable` in `subjects/choices`, merged in here for completeness; if not already present in `analysis_data`, join it from `subjects` first.

```
wilcox.test(Age ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Age by NonDelegator
## W = 2458.5, p-value = 0.6487
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  TimeInSeconds by NonDelegator
## W = 1833, p-value = 0.1052
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(RiskTaking ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  RiskTaking by NonDelegator
## W = 2265.5, p-value = 0.854
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(AIXperience ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  AIXperience by NonDelegator
## W = 2550, p-value = 0.445
## alternative hypothesis: true location shift is not equal to 0
```

```
# Reliability (Reliable), if not yet merged into analysis_data:
if (!"Reliable" %in% names(analysis_data)) {
  analysis_data <- analysis_data %>%
    left_join(subjects %>% select(ProlificID, Reliable), by = "ProlificID")
```

```
}
t.test(Reliable ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  Reliable by NonDelegator
## t = -0.058833, df = 25.559, p-value = 0.9535
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.1394700  0.1317147
## sample estimates:
## mean in group 0 mean in group 1
##        6.905213        6.909091
```

```
wilcox.test(Reliable ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Reliable by NonDelegator
## W = 2312, p-value = 0.9555
## alternative hypothesis: true location shift is not equal to 0
```

5.3 Choice quality among never-delegators (new Limitations check)

Tests whether never-delegators' own non-dominated choice rate (among their self-made Part-2 decisions) differs from that of participants who delegated at least once, used in the manuscript to argue against a task-difficulty confound for categorical non-delegation (paper: 34.7% vs. 33.6%; Welch $t = -0.25$, $p = 0.806$; Wilcoxon $p = 0.669$).

```
selfchoice <- choices %>%
  filter(Part == 2, Delegated == 0, Option > 0) %>%
  left_join(
    analysis_data %>% select(ProlificID, DelegatorType),
    by = "ProlificID"
  )

# Decision-level check, by delegator type
selfchoice %>%
  group_by(DelegatorType) %>%
  summarise(avNonDom = mean(Nondominated, na.rm = TRUE), n = n(), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   DelegatorType      avNonDom     n
##   <fct>                 <dbl> <int>
## 1 NeverDelegator        0.349   175
## 2 SometimesDelegator    0.345   753
```

```
# Participant-level average non-dominated rate, Never vs. AtLeastOnce
partavg <- selfchoice %>%
  group_by(ProlificID) %>%
  summarise(Nondominated = mean(Nondominated, na.rm = TRUE), .groups = "drop") %>%
  left_join(analysis_data %>% select(ProlificID, DelegationGroup), by = "ProlificID")
```

```r
partavg %>%
  group_by(DelegationGroup) %>%
  summarise(Nondominated = mean(Nondominated, na.rm = TRUE), n = n(), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   DelegationGroup Nondominated     n
##   <chr>                  <dbl> <int>
## 1 AtLeastOnce            0.336   170
## 2 Never                  0.347    22
```

```r
t.test(Nondominated ~ DelegationGroup, data = partavg)
```

```
##
##  Welch Two Sample t-test
##
## data:  Nondominated by DelegationGroup
## t = -0.248, df = 34.608, p-value = 0.8056
## alternative hypothesis: true difference in means between group AtLeastOnce and group Never is not eq
## 95 percent confidence interval:
##  -0.09505609  0.07436727
## sample estimates:
## mean in group AtLeastOnce       mean in group Never
##                 0.3362465                 0.3465909
```

```r
wilcox.test(Nondominated ~ DelegationGroup, data = partavg)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Nondominated by DelegationGroup
## W = 1765.5, p-value = 0.6689
## alternative hypothesis: true location shift is not equal to 0
```

```r
# Within the preference-based condition: Part-1 satisfaction by delegation type
NeverDelegate <- choices %>%
  select(ProlificID, sumDelegations) %>%
  distinct() %>%
  mutate(sumDeleg0 = (sumDelegations == 0))

subjects2 <- subjects %>% left_join(NeverDelegate, by = "ProlificID")

subjects2 %>%
  filter(EVAlgorithm == 0) %>%
  group_by(sumDeleg0) %>%
  summarise(SatisfiedPart1 = mean(SatisfiedPart1, na.rm = TRUE),
            n = n(), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   sumDeleg0 SatisfiedPart1     n
##   <lgl>              <dbl> <int>
## 1 FALSE               7.56    97
## 2 TRUE                7.94    18
```

6 Responses to feedback (win-stay / lose-shift)

Win-stay/lose-shift model on rounds 2–8 (paper, Section "Responses to feedback").

```
choices <- choices %>%
  group_by(ProlificID) %>%
  arrange(WithinPartDecisionNumber, .by_group = TRUE) %>%
  mutate(
    lagDelegated          = lag(Delegated, 1),
    lagRelevantPayoff     = lag(RelevantPayoff, 1),
    lagSatisfactionRating = lag(SatisfactionRating, 1)
  ) %>%
  ungroup()

choices$stay <- 1 * (choices$Delegated == choices$lagDelegated)

# "Prior win": the outcome of the PREVIOUS round was strictly positive.
# (lagWin is defined from the lagged payoff, i.e. the genuine win-stay/lose-shift
# notion of reacting to the previously experienced outcome.)
choices$lagWin <- 1 * (choices$lagRelevantPayoff > 0)

# Pooled model: treatment x prior win
winStayReg <- feols(
  stay ~ EVAlgorithm * lagWin | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg)
```

```
## OLS estimation, Dep. Var.: stay
## Observations: 1,615
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                     Estimate Std. Error   t value   Pr(>|t|)
## EVAlgorithm        -0.104670   0.049740 -2.104353 3.6424e-02 *
## lagWin              0.169470   0.035178  4.817493 2.6258e-06 ***
## EVAlgorithm:lagWin  0.008762   0.048728  0.179822 8.5745e-01
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.456921     Adj. R2: 0.052558
##                  Within R2: 0.043757
```

Separately by treatment, distinguishing whether the previous round was delegated (`lagDelegated == 1`, i.e. "the algorithm" produced the outcome) from a self-made choice. This separates lose-shift after one's own loss from lose-shift after a delegated (algorithmic) loss.

```
# EV condition
winStayReg_EV <- feols(
  stay ~ lagDelegated * I(1 - lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVAlgorithm == 1 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg_EV)
```

```
## OLS estimation, Dep. Var.: stay
```

```
## Observations: 819
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                              Estimate Std. Error   t value  Pr(>|t|)
## lagDelegated                 0.044647   0.044771  0.997214 0.3207185
## I(1 - lagWin)               -0.075637   0.050734 -1.490836 0.1386957
## lagDelegated:I(1 - lagWin) -0.184727   0.069840 -2.645020 0.0092905 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.473051     Adj. R2: 0.03568
##                  Within R2: 0.038964
```

```
# Preference-based condition
winStayReg_Pref <- feols(
  stay ~ lagDelegated * I(1 - lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVAlgorithm == 0 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg_Pref)
```

```
## OLS estimation, Dep. Var.: stay
## Observations: 796
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                              Estimate Std. Error   t value Pr(>|t|)
## lagDelegated                 0.024197   0.036355  0.665574 0.507028
## I(1 - lagWin)               -0.108747   0.048066 -2.262433 0.025565 *
## lagDelegated:I(1 - lagWin) -0.140381   0.063708 -2.203504 0.029570 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.432604     Adj. R2: 0.058073
##                  Within R2: 0.042744
```

```
# Average stay rates by condition, prior delegation, and prior win/loss
choices %>%
  filter(Part == 2, WithinPartDecisionNumber > 1) %>%
  group_by(EVAlgorithm, lagDelegated, lagWin) %>%
  summarise(avg_stay_rate = mean(stay, na.rm = TRUE),
            n = n(), .groups = "drop") %>%
  arrange(EVAlgorithm, lagDelegated, lagWin)
```

```
## # A tibble: 9 x 5
##   EVAlgorithm lagDelegated lagWin avg_stay_rate     n
##         <int>        <int>  <dbl>         <dbl> <int>
## 1           0            0      0         0.677   155
## 2           0            0      1         0.769   255
## 3           0            1      0         0.547   139
## 4           0            1      1         0.785   247
## 5           0           NA     NA       NaN         2
## 6           1            0      0         0.583   144
## 7           1            0      1         0.668   262
## 8           1            1      0         0.444   169
## 9           1            1      1         0.697   244
```

7 Exploratory mechanisms

7.1 Coefficient model (interaction with performance expectations)

LPM of individual delegation decisions interacting treatment with participants' algorithmic performance expectations, controlling for round, per-choice Part-1 confidence, revealed Part-1 risk preferences, and task fixed effects.

```
delRegWConf <- feols(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + ConfidenceP1 +
    ProbOfAbsurdComputerChoices + ProbOfBadComputerChoices +
    EVChoices + truePChoices) | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegWConf)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,848
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                          Estimate Std. Error   t value
## EVAlgorithm                              0.856469   0.891957  0.960213
## WithinPartDecisionNumber                -0.009920   0.005711 -1.737089
## ConfidenceP1                            -0.025275   0.012620 -2.002676
## ProbOfAbsurdComputerChoices              0.009864   0.018425  0.535348
## ProbOfBadComputerChoices                -0.060048   0.016324 -3.678547
## EVChoices                                0.067083   0.058904  1.138847
## truePChoices                             0.058960   0.054107  1.089695
## EVAlgorithm:WithinPartDecisionNumber    -0.012262   0.008341 -1.470058
## EVAlgorithm:ConfidenceP1                 0.043682   0.015740  2.775229
## EVAlgorithm:ProbOfAbsurdComputerChoices -0.072908   0.023059 -3.161780
## EVAlgorithm:ProbOfBadComputerChoices     0.036186   0.020603  1.756318
## EVAlgorithm:EVChoices                   -0.089742   0.077330 -1.160496
## EVAlgorithm:truePChoices                -0.095940   0.071561 -1.340675
##                                          Pr(>|t|)
## EVAlgorithm                             0.3379475
## WithinPartDecisionNumber                0.0836987 .
## ConfidenceP1                            0.0463759 *
## ProbOfAbsurdComputerChoices             0.5929221
## ProbOfBadComputerChoices                0.0002916 ***
## EVChoices                               0.2559415
## truePChoices                            0.2769783
## EVAlgorithm:WithinPartDecisionNumber    0.1429009
## EVAlgorithm:ConfidenceP1                0.0059660 **
## EVAlgorithm:ProbOfAbsurdComputerChoices 0.0017772 **
## EVAlgorithm:ProbOfBadComputerChoices    0.0803535 .
## EVAlgorithm:EVChoices                   0.2470398
## EVAlgorithm:truePChoices                0.1813374
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.474486     Adj. R2: 0.089466
##                  Within R2: 0.089764
```

7.2 Treatment comparison of expected algorithmic errors

Formal comparison of expected non-preferred and completely weird choices across treatments. A LOWER expected number of non-preferred choices under the preference-based algorithm would indicate that participants believe the personalized algorithm is doing its job (matching their preferences).

```
expect_tbl <- subjects %>%
  mutate(Condition = ifelse(EVAlgorithm == 1, "EV", "Preference-based")) %>%
  group_by(Condition) %>%
  summarise(
    n                  = n(),
    mean_nonpreferred  = mean(ProbOfBadComputerChoices, na.rm = TRUE),
    sd_nonpreferred    = sd(ProbOfBadComputerChoices, na.rm = TRUE),
    mean_weird         = mean(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    sd_weird           = sd(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    .groups = "drop"
  )
expect_tbl
```

```
## # A tibble: 2 x 6
##   Condition            n mean_nonpreferred sd_nonpreferred mean_weird sd_weird
##   <chr>            <int>             <dbl>           <dbl>      <dbl>    <dbl>
## 1 EV                 118              5               2.32       1.78     1.74
## 2 Preference-based   115              3.78            2.16       1.57     1.62
```

```
# Expected NON-PREFERRED choices: EV vs. Preference-based
t.test(ProbOfBadComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfBadComputerChoices by EVAlgorithm
## t = -4.1504, df = 230.54, p-value = 4.671e-05
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -1.7953130 -0.6394696
## sample estimates:
## mean in group 0 mean in group 1
##        3.782609        5.000000
```

```
wilcox.test(ProbOfBadComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  ProbOfBadComputerChoices by EVAlgorithm
## W = 4805, p-value = 9.898e-05
## alternative hypothesis: true location shift is not equal to 0
```

```
# Expected COMPLETELY WEIRD choices: EV vs. Preference-based
t.test(ProbOfAbsurdComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfAbsurdComputerChoices by EVAlgorithm
```

```
## t = -0.93521, df = 230.48, p-value = 0.3507
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.6392193  0.2277233
## sample estimates:
## mean in group 0 mean in group 1
##        1.573913        1.779661
```

```
wilcox.test(ProbOfAbsurdComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  ProbOfAbsurdComputerChoices by EVAlgorithm
## W = 6328, p-value = 0.361
## alternative hypothesis: true location shift is not equal to 0
```

```
# Within the preference-based condition: do never-delegators expect the
# personalized algorithm to make MORE non-preferred choices than delegators?
# (If not, categorical non-delegation is not driven by distrust in the
#  algorithm's ability to capture one's preferences.)
analysis_data %>%
  filter(EVAlgorithm == 0) %>%
  group_by(NonDelegator) %>%
  summarise(
    n                 = n(),
    mean_nonpreferred = mean(ProbOfBadComputerChoices, na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 3
##   NonDelegator     n mean_nonpreferred
##          <dbl> <int>             <dbl>
## 1            0    97              3.77
## 2            1    18              3.83
```

```
t.test(ProbOfBadComputerChoices ~ NonDelegator,
       data = subset(analysis_data, EVAlgorithm == 0))
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfBadComputerChoices by NonDelegator
## t = -0.11119, df = 24.369, p-value = 0.9124
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -1.175546  1.055272
## sample estimates:
## mean in group 0 mean in group 1
##        3.773196        3.833333
```

8 Post-task reasons

8.1 Figure 4 — Reasons by treatment and delegation type

Reason-selection frequencies for the hypothetical additional-task choice, by treatment (EV vs. preference-based) and delegation type (delegators vs. never-delegators).

```
subjects2 <- subjects %>% left_join(NeverDelegate, by = "ProlificID")

subjects_long <- subjects2 %>%
  pivot_longer(cols = DontTrust:Other, names_to = "item", values_to = "clicked")

# Group sizes for legend labels
n_labels <- subjects_long %>%
  group_by(sumDeleg0, EVAlgorithm) %>%
  summarise(n = n_distinct(ProlificID), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE  & EVAlgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE  & EVAlgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 1 ~ "EV--Delegators"
    ),
    group_label = paste0(group, " (n=", n, ")")
  )

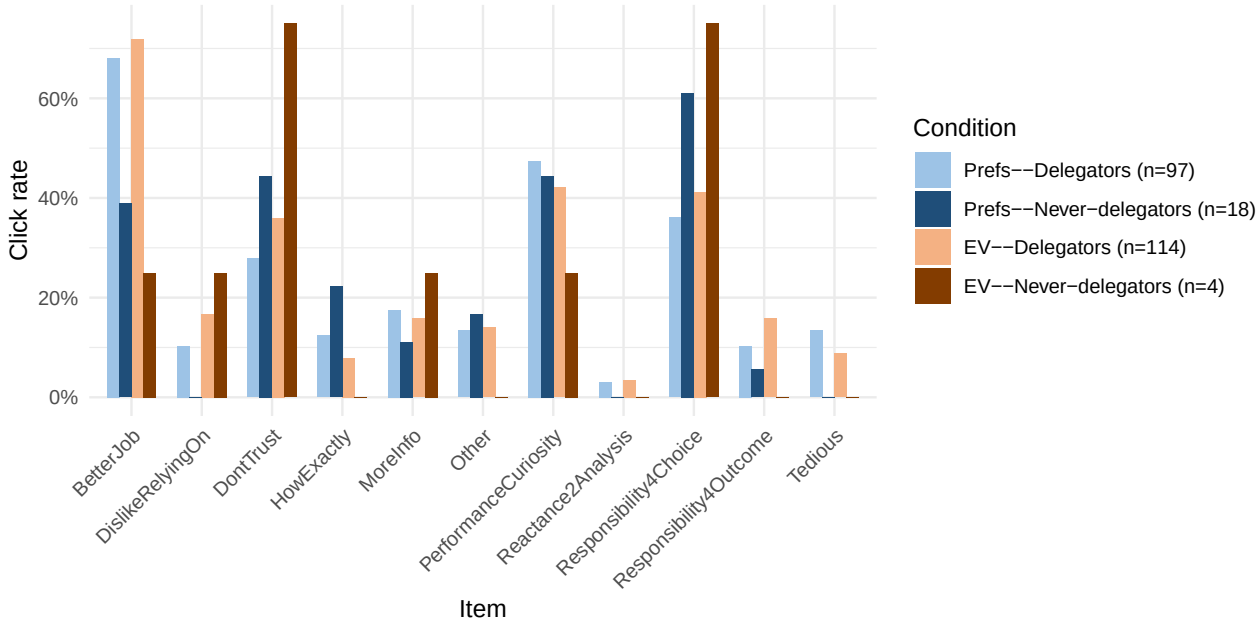
group_colors <- c(
  "Prefs--Delegators"       = "#9dc3e6",
  "Prefs--Never-delegators" = "#1f4e79",
  "EV--Delegators"          = "#f4b183",
  "EV--Never-delegators"    = "#833c00"
)
label_map <- setNames(n_labels$group_label, n_labels$group)

click_rates_both <- subjects_long %>%
  group_by(sumDeleg0, EVAlgorithm, item) %>%
  summarise(rate = mean(clicked, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE  & EVAlgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE  & EVAlgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 1 ~ "EV--Delegators"
    ),
    group = factor(group, levels = names(group_colors))
  )

fig4 <- ggplot(click_rates_both, aes(x = item, y = rate, fill = group)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.7) +
  scale_fill_manual(values = group_colors, labels = label_map,
                    breaks = names(group_colors), name = "Condition") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(x = "Item", y = "Click rate") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
```

```
        legend.position = "right")

fig4
```




```
ggsave("Figure4_ReasonsByTrmtAndNeverDelegator2.pdf", fig4, width = 8, height = 4, dpi =
↪  300)
```

8.2 Aggregate reason patterns

In-text figures for performance, distrust, and responsibility motives by delegation type (delegators vs. never-delegators).

```
subjects_long %>%
  group_by(sumDeleg0, item) %>%
  summarise(rate = mean(clicked, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = sumDeleg0, values_from = rate) %>%
  rename(Delegators = `FALSE`, NeverDelegators = `TRUE`)
```

```
## # A tibble: 11 x 3
##    item                   Delegators NeverDelegators
##    <chr>                       <dbl>           <dbl>
##  1 BetterJob                  0.701           0.364
##  2 DislikeRelyingOn           0.137           0.0455
##  3 DontTrust                  0.322           0.5
##  4 HowExactly                 0.0995          0.182
##  5 MoreInfo                   0.166           0.136
##  6 Other                      0.137           0.136
##  7 PerformanceCuriosity       0.445           0.409
##  8 Reactance2Analysis         0.0332          0
##  9 Responsibility4Choice      0.389           0.636
## 10 Responsibility4Outcome     0.133           0.0455
## 11 Tedious                    0.109           0
```

8.3 LPM: never-delegation on reasons

```
reason_vars <- c("DontTrust", "BetterJob", "HowExactly", "MoreInfo",
                 "Responsibility4Choice", "Responsibility4Outcome",
                 "DislikeRelyingOn", "Reactance2Analysis",
                 "PerformanceCuriosity", "Tedious", "Other")

# Bivariate LPMs (one per reason, HC2 SEs)
bivariate_tbl <- do.call(rbind, lapply(reason_vars, function(rv) {
  m  <- lm(as.formula(paste("as.numeric(sumDeleg0) ~", rv)), data = subjects2)
  ct <- coeftest(m, vcov = vcovHC(m, type = "HC2"))
  data.frame(reason = rv,
             coef = round(ct[2, 1], 3), se = round(ct[2, 2], 3),
             t = round(ct[2, 3], 3), p = round(ct[2, 4], 3))
}))
print(bivariate_tbl, row.names = FALSE)
```

```
##                  reason   coef    se      t     p
##               DontTrust  0.068 0.044  1.528 0.128
##               BetterJob -0.131 0.048 -2.739 0.007
##              HowExactly  0.073 0.077  0.950 0.343
##                MoreInfo -0.018 0.049 -0.376 0.707
##   Responsibility4Choice  0.087 0.041  2.111 0.036
##  Responsibility4Outcome -0.068 0.041 -1.688 0.093
##        DislikeRelyingOn -0.070 0.040 -1.769 0.078
##      Reactance2Analysis -0.097 0.020 -4.926 0.000
##    PerformanceCuriosity -0.013 0.038 -0.328 0.743
##                 Tedious -0.105 0.021 -4.945 0.000
##                   Other -0.001 0.056 -0.014 0.989
```

```
# Joint LPM: full sample + preference-treatment only
f_joint  <- as.formula(paste("as.numeric(sumDeleg0) ~",
                             paste(reason_vars, collapse = " + ")))
lpm_all  <- lm(f_joint, data = subjects2)
lpm_pref <- lm(f_joint, data = subjects2[subjects2$EVAlgorithm == 0, ])

vcov_all  <- vcovHC(lpm_all,  type = "HC2")
vcov_pref <- vcovHC(lpm_pref, type = "HC2")

screenreg(
  list(lpm_all, lpm_pref),
  override.se = list(sqrt(diag(vcov_all)), sqrt(diag(vcov_pref))),
  override.pvalues = list(
    2 * pt(abs(coef(lpm_all)  / sqrt(diag(vcov_all))),
           df = lpm_all$df.residual,  lower.tail = FALSE),
    2 * pt(abs(coef(lpm_pref) / sqrt(diag(vcov_pref))),
           df = lpm_pref$df.residual, lower.tail = FALSE)
  ),
  custom.model.names = c("Full sample", "Pref. treatment"),
  caption = "LPM: never-delegation ~ reasons (HC2 SEs)"
)
```

```
##
## ================================================
```

```
##                          Full sample  Pref. treatment
## ---------------------------------------------------
## (Intercept)               0.14 **      0.20 *
##                          (0.05)       (0.09)
## DontTrust                 0.08         0.15
##                          (0.04)       (0.09)
## BetterJob                -0.11 *      -0.14
##                          (0.05)       (0.08)
## HowExactly                0.04         0.05
##                          (0.08)       (0.10)
## MoreInfo                 -0.04        -0.15
##                          (0.06)       (0.11)
## Responsibility4Choice     0.06         0.11
##                          (0.04)       (0.07)
## Responsibility4Outcome   -0.04        -0.00
##                          (0.04)       (0.10)
## DislikeRelyingOn         -0.09 *      -0.25 **
##                          (0.04)       (0.08)
## Reactance2Analysis       -0.09        -0.12
##                          (0.04)       (0.12)
## PerformanceCuriosity      0.02         0.03
##                          (0.04)       (0.07)
## Tedious                  -0.07 **     -0.16 **
##                          (0.03)       (0.06)
## Other                    -0.02        -0.01
##                          (0.06)       (0.12)
## ---------------------------------------------------
## R^2                       0.10         0.16
## Adj. R^2                  0.05         0.06
## Num. obs.                233          115
## ===================================================
## *** p < 0.001; ** p < 0.01; * p < 0.05
```

9 Exclusion-criterion breakdown (Method section)

Reconstructs the exact breakdown of the 7 excluded participants by criterion, as reported in the Method section (paper: 1 bot/slider failure, 4 careful-participation failures [`Reliable < 6`], 3 disclosed-AI-usage cases, 0 comprehension-only failures, 0 incomplete-data cases, 0 log-time exclusions; final $N = 233$, with 1 participant overlapping careful-participation-failure and AI-disclosure). This chunk re-reads the raw files independently of the exclusions already applied above (using `_raw` suffixes), so it is self-contained and can be run at any point in the script.

```
choices_raw      <- read.delim("260206_1528_537_lotteryChoices.xls")
prolificData_raw <- read.csv("prolific_demographic_export_approved.csv")

cat("N recruited (Prolific export):", nrow(prolificData_raw), "\n")
```

```
## N recruited (Prolific export): 240
```

```
choices_raw <- subset(choices_raw, ProlificID %in% prolificData_raw$Participant.id)

part2_raw <- subset(choices_raw, Part == 2)
partvars <- part2_raw[!duplicated(part2_raw$ProlificID),
                      c("ProlificID", "BotCheck1attempt", "Part1CQsPassedOnAttempt",
                        "Part2CQsPassedOnAttempt", "Reliable", "AIUsageInStudy")]

# Incomplete-data flag: participant has zero valid (Delegated >= 0) Part-2 rows
valid_counts <- aggregate(Delegated ~ ProlificID, data = part2_raw,
                          FUN = function(x) sum(x >= 0))
names(valid_counts)[2] <- "nValid"
partvars <- merge(partvars, valid_counts, by = "ProlificID", all.x = TRUE)
partvars$incomplete <- is.na(partvars$nValid) | partvars$nValid == 0

# Log-completion-time "too fast" rule
prolificData_raw$logCompleTime <- log(prolificData_raw$Time.taken)
prolificData_raw$tooFast <- 1 * (
  prolificData_raw$logCompleTime <
    mean(log(prolificData_raw$Time.taken)) - 2 * sd(log(prolificData_raw$Time.taken))
)
partvars <- merge(partvars, prolificData_raw[, c("Participant.id", "tooFast")],
                  by.x = "ProlificID", by.y = "Participant.id", all.x = TRUE)

partvars$failBot      <- partvars$BotCheck1attempt != 1
partvars$failComp     <- partvars$Part1CQsPassedOnAttempt > 2 |
                         partvars$Part2CQsPassedOnAttempt > 2
partvars$failCareful  <- partvars$Reliable < 6
partvars$disclosedAI  <- partvars$AIUsageInStudy != 0

excluded <- subset(
  partvars,
  failBot | failComp | failCareful | disclosedAI | incomplete | tooFast == 1
)

cat("\nTotal excluded:", nrow(excluded), "\n")
```

```
##
## Total excluded: 7
```

```r
cat("Fail bot/slider check:", sum(excluded$failBot), "\n")
```

```
## Fail bot/slider check: 1
```

```r
cat("Fail comprehension check only:", sum(excluded$failComp), "\n")
```

```
## Fail comprehension check only: 0
```

```r
cat("Fail careful-participation check (Reliable < 6):", sum(excluded$failCareful), "\n")
```

```
## Fail careful-participation check (Reliable < 6): 4
```

```r
cat("Disclosed AI usage:", sum(excluded$disclosedAI), "\n")
```

```
## Disclosed AI usage: 3
```

```r
cat("Incomplete data:", sum(excluded$incomplete), "\n")
```

```
## Incomplete data: 0
```

```r
cat("Too fast (log-time rule):", sum(excluded$tooFast == 1, na.rm = TRUE), "\n")
```

```
## Too fast (log-time rule): 0
```

```r
cat("Overlap (careful-participation AND disclosed AI):",
    sum(excluded$failCareful & excluded$disclosedAI), "\n")
```

```
## Overlap (careful-participation AND disclosed AI): 1
```

```r
cat("Final analysis N:", nrow(partvars) - nrow(excluded), "\n")
```

```
## Final analysis N: 233
```

10 Power analysis (pilot-based, new Limitations paragraph)

Reconstructs the ex-ante power analysis reported as a new Limitations paragraph: using only the pilot's effect sizes (not the present study's own results, to avoid the retrospective-power fallacy; Hoenig & Heisey, 2001), we compute the minimum detectable effect (MDE) at 80% power for $n = 100$–120/arm, and the power to re-detect an effect of the pilot's observed magnitude. Pilot data: `pilot_data_complete_iw.csv` (an earlier, non-preregistered study reported in Suraci's PhD thesis).

```
pilot <- read.csv("pilot_data_complete_iw.csv")
pilot <- subset(pilot, returned == 1)
cat("Pilot N (returned == 1):", nrow(pilot), "\n")
```

```
## Pilot N (returned == 1): 377
```

```
# EV treatments = 1,2; preference-based (RDU) treatments = 3,4
pilot$EValgo <- 1 * (pilot$treatment %in% c(1, 2))
cat("\nPilot N by condition (0 = Pref, 1 = EV):\n")
```

```
##
## Pilot N by condition (0 = Pref, 1 = EV):
```

```
print(table(pilot$EValgo))
```

```
##
##   0   1
## 190 187
```

```
# First-contact delegation (randomized-order "1st" decision)
cat("\nFirst-contact delegation rate by condition (paper: 45.8% Pref vs. 67.4% EV):\n")
```

```
##
## First-contact delegation rate by condition (paper: 45.8% Pref vs. 67.4% EV):
```

```
print(aggregate(decision1st ~ EValgo, data = pilot, mean))
```

```
##   EValgo decision1st
## 1      0   0.4578947
## 2      1   0.6737968
```

```
# Average delegation across all 8 (randomized-order) decisions
decisioncols <- c("decision1st", "decision2nd", "decision3rd", "decision4th",
                  "decision5th", "decision6th", "decision7th", "decision8th")
pilot$avgDeleg <- rowMeans(pilot[, decisioncols], na.rm = TRUE)
cat("\nAverage delegation rate by condition (paper: 35.3% Pref vs. 54.5% EV):\n")
```

```
##
## Average delegation rate by condition (paper: 35.3% Pref vs. 54.5% EV):
```

```
print(aggregate(avgDeleg ~ EValgo, data = pilot, mean))
```

```
##   EValgo  avgDeleg
## 1      0 0.3532895
## 2      1 0.5447861
```

10.1 Minimum detectable effect (MDE) at 80% power

A pure design-stage quantity: depends only on n, α, and a baseline rate, not on anything observed in the present study (paper: ~16-17pp for first contact, ~10-11pp for average delegation).

```r
p_pref_firstcontact <- aggregate(decision1st ~ EValgo, data = pilot, mean)[1, 2]

for (n in c(100, 120)) {
  pt <- power.prop.test(n = n, p1 = p_pref_firstcontact, p2 = NULL,
                        power = 0.80, sig.level = 0.05, alternative = "one.sided")
  cat("n =", n, " MDE (first-contact-type proportion) =",
      round(pt$p2 - p_pref_firstcontact, 4), "\n")
}
```

```
## n = 100  MDE (first-contact-type proportion) = 0.1742
## n = 120  MDE (first-contact-type proportion) = 0.1594
```

10.2 MDE and power for the average-delegation comparison

Approximated with a one-sided two-sample t-test using the pilot's pooled within-condition SD of participant-average delegation (paper: MDE ~10-11pp; 99.7-99.9% power to re-detect the pilot's ~19pp gap).

```r
sd_pilot    <- sqrt(mean(tapply(pilot$avgDeleg, pilot$EValgo, var)))
delta_pilot <- diff(tapply(pilot$avgDeleg, pilot$EValgo, mean))
cat("Pilot pooled SD:", round(sd_pilot, 4), " pilot gap:", round(delta_pilot, 4), "\n")
```

```
## Pilot pooled SD: 0.3085  pilot gap: 0.1915
```

```r
for (n in c(100, 120)) {
  m <- power.t.test(n = n, sd = sd_pilot, power = 0.80, sig.level = 0.05,
                    alternative = "one.sided")
  r <- power.t.test(n = n, delta = delta_pilot, sd = sd_pilot, sig.level = 0.05,
                    alternative = "one.sided")
  cat("n =", n, " MDE =", round(m$delta, 4), " power to re-detect pilot gap =",
      round(r$power, 4), "\n")
}
```

```
## n = 100  MDE = 0.1089  power to re-detect pilot gap = 0.9968
## n = 120  MDE = 0.0993  power to re-detect pilot gap = 0.9992
```

10.3 Power to re-detect the pilot's observed effect

Not a retrospective-power computation on the present study's own effect (which would be circular); this uses only the pilot's pre-existing effect size as input (paper: 93-96% under the normal approximation, 91-95% by exact-test simulation).

```r
p_ev_firstcontact <- aggregate(decision1st ~ EValgo, data = pilot, mean)[2, 2]

for (n in c(100, 120)) {
  pt <- power.prop.test(n = n, p1 = p_pref_firstcontact, p2 = p_ev_firstcontact,
                        sig.level = 0.05, alternative = "one.sided")
  cat("n =", n, " power to re-detect pilot's first-contact gap =",
      round(pt$power, 4), "\n")
}
```

```
## n = 100  power to re-detect pilot's first-contact gap = 0.9293
## n = 120  power to re-detect pilot's first-contact gap = 0.9618
```

```r
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
```

```
## Platform: x86_64-pc-linux-gnu
## Running under: TUXEDO OS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=de_CH.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=de_CH.UTF-8        LC_COLLATE=de_CH.UTF-8
##  [5] LC_MONETARY=de_CH.UTF-8    LC_MESSAGES=de_CH.UTF-8
##  [7] LC_PAPER=de_CH.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=de_CH.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Europe/Zurich
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] texreg_1.39.5  sandwich_3.1-1 lmtest_0.9-40  zoo_1.8-15     fixest_0.14.1
##  [6] Exact_3.3      scales_1.4.0   ggplot2_4.0.3  tidyr_1.3.2    dplyr_1.2.1
##
## loaded via a namespace (and not attached):
##  [1] utf8_1.2.6          generics_0.1.4      dreamerr_1.5.0
##  [4] lattice_0.23-1      digest_0.6.39       magrittr_2.0.5
##  [7] evaluate_1.0.5      grid_4.6.1          RColorBrewer_1.1-3
## [10] fastmap_1.2.0       Formula_1.2-5       tinytex_0.59
## [13] httr_1.4.8          purrr_1.2.2         stringmagic_1.2.0
## [16] numDeriv_2016.8-1.1 textshaping_1.0.5   cli_3.6.6
## [19] rlang_1.2.0         withr_3.0.2         yaml_2.3.12
## [22] rootSolve_1.8.2.4   tools_4.6.1         vctrs_0.7.3
## [25] R6_2.6.1            lifecycle_1.0.5     ragg_1.5.2
## [28] pkgconfig_2.0.3     pillar_1.11.1       gtable_0.3.6
## [31] glue_1.8.1          Rcpp_1.1.1-1.1      systemfonts_1.3.2
## [34] xfun_0.57           tibble_3.3.1        tidyselect_1.2.1
## [37] rstudioapi_0.18.0   knitr_1.51          farver_2.1.2
## [40] htmltools_0.5.9     nlme_3.1-171        rmarkdown_2.31
## [43] labeling_0.4.3      compiler_4.6.1      S7_0.2.2
```